Assignment-4

Medina Alish, SE-2538

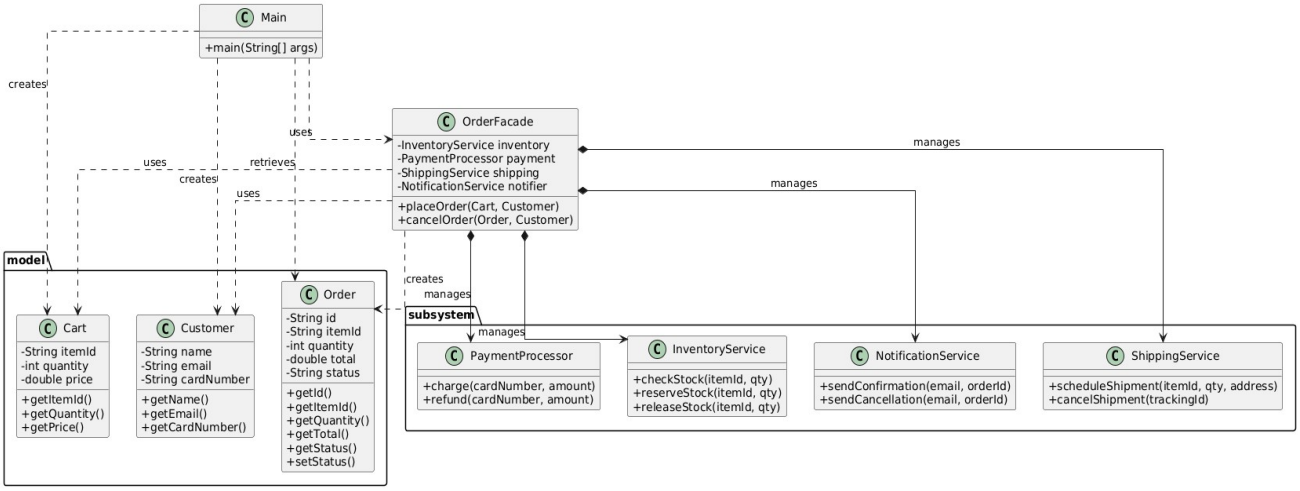
Repository Link: aalishmee/sdp4

1. Introduction

For this assignment, I implemented the Facade pattern using the Online Order Processing domain (Option B). Placing an online order requires a strict sequence of operations: checking inventory, charging the customer, scheduling shipping, and sending a confirmation email.

If a client had to call all four of these services directly, it would tightly couple the client to the subsystem. Furthermore, the client would be forced to handle complex rollback logic (e.g., releasing reserved stock if a payment fails). The OrderFacade solves this problem by providing a single, simple method (placeOrder) that orchestrates the entire process internally. It ensures the correct order of operations and handles error recovery automatically, leaving the client with a clean and simple API.

2. UML Class Diagram



3. Clean Code Principles

I identified and applied the following five Clean Code principles in my implementation:

1. Meaningful Names

The Facade exposes methods like placeOrder(cart, customer) and cancelOrder(order, customer). These names state exactly what business intent they fulfill. The client does not need to know about checking stock, charging cards, or scheduling shipments-the names hide the technical steps behind clear, domain-specific language.

2. Single Responsibility Principle (SRP)

The OrderFacade only coordinates the process; it does not reimplement subsystem logic.
Before (Bad): The Facade would calculate total = price * qty and validate the card format.
After (Good): The Cart class holds the data, PaymentProcessor handles the charge, and the Facade simply says "charge this amount."

3. No Leaking of Subsystem Details

The client (Main) never sees InventoryService or PaymentProcessor. The Facade's public API only returns an Order object or throws an exception.
Before (Bad): public InventoryService getInventory() { return inventory; } - this allows the client to bypass the Facade.
After (Good): The subsystem fields are private final and are never exposed via getters.

4. Small, Focused Subsystem Classes
Each subsystem class has one clear job. InventoryService only handles stock, PaymentProcessor only handles money, and NotificationService only sends emails. They do not know about each other. This makes the code easy to test and maintain independently.

5. Proper Error Handling for Coordination Failures
If a step fails mid-process, the Facade handles the rollback cleanly. In the placeOrder method, if PaymentProcessor.charge() returns false, the Facade catches this failure, calls inventory.releaseStock() to roll back the reservation, and throws a clean IllegalStateException to the client. The client doesn't need to write try-catch blocks for every single subsystem step.

```
27          if (!inventory.checkStock(itemId, qty)) {
28              throw new IllegalStateException("Out of stock: " + itemId);
29          }
30          inventory.reserveStock(itemId, qty);
31
32          if (!payment.charge(customer.getCardNumber(), total)) {
33              inventory.releaseStock(itemId, qty);
34              throw new IllegalStateException("Payment declined");
35          }
36
37          String tracking = shipping.scheduleShipment(
38                  itemId, qty, customer.getEmail());
39          System.out.println("[Facade] Shipment tracking ID: " + tracking);
```

4. Conclusion

The Facade pattern successfully hides the complex sequence of subsystem calls and the critical rollback logic required for failed transactions. It provides a much-needed simplified interface (OrderFacade) for the client.

However, a defining property of this pattern is that the subsystem remains completely accessible. The subsystem classes (InventoryService, PaymentProcessor, ShippingService, NotificationService) do not have any back-references to the Facade—they do not even know it exists. They are declared as public classes and can still be instantiated and used directly by advanced clients who wish to bypass the Facade for custom, low-level operations.

```
[Inventory] Checking stock for BOOK-67
[Inventory] Reserved 2 x BOOK-67
[Payment] Charging $30.0
[Shipping] Scheduled 2 x BOOK-67 to aruzhan@mail.com (TRK-1792342414047)
[Facade] Shipment tracking ID: TRK-1792342414047
[Notify] Confirmation sent to aruzhan@mail.com for order ORD-1792342414058
Order placed: ORD-1792342414058
[Inventory] Checking stock for TV-67
[Inventory] Reserved 1 x TV-67
[Payment] Charging $99999.0
[Inventory] Released 1 x TV-67
Error! Payment declined (stock was released)
```